

Edge Language Coach — Architecture Report

Scalable & Reliable Services

Contents

1. System Overview	1
2. Reliability	2
2.0 Service Level Objectives	2
2.1 Reliability primitives	3
2.2 Circuit breaker measured behaviour	3
2.3 Other primitives	3
2.4 Recovery Time Objectives	4
2.5 Tests in CI	4
3. Scalability	4
3.1 Mechanism	4
3.2 Measured scale-out behaviour	5
3.3 Threats to validity	5
3.4 Production deployment	5
4. Observability	6
5. Operator Surface — Agentic MCP	6
5.1 Capability classification	7
5.2 Guardrails	8
6. Safety policy and cost analysis	8
7. Trade-offs	9
8. Future Work	9

1. System Overview

Edge Language Coach is a small web app for practising Italian. You chat with an LLM tutor, the system scrapes RSS feeds in the background to keep conversation topics fresh and each session is turned into spaced-repetition flashcards afterwards.

We built it as three independently runnable services behind a load balancer, with Supabase for the database and auth and Groq for the LLM calls.

Unit	Technology	Role
Web	React 19 + Vite	SPA, served by Vercel in production and nginx in local dev
Gateway	Fastify 5 (Node.js 22)	REST API, auth, reliability primitives
Workers	Node.js + BullMQ	Async jobs (scraper, summary, flashcard)
Redis	redis:7-alpine	BullMQ queues and distributed rate-limit counters
Observability	Prometheus + Grafana	Metrics scraping and dashboards (local dev only)

The gateway holds no session state in memory. Every request carries a Supabase JWT and every rate-limit counter lives in Redis. That is what makes horizontal scale-out a one-line change later (Section 3).

Request paths differ between environments. In local dev:

Browser → web nginx (:80) → nginx-lb (:3001) → gateway:N → Redis | Supabase | Groq

In production, the SPA is on Vercel and Vercel rewrites `/api/*` server-side to the Oracle backend:

Browser → Vercel (SPA + `/api` rewrite) → nginx-lb (:3001) → gateway:N → Redis | Supabase | Groq

The async path is the same in both:

gateway → Redis (BullMQ queue) → workers → Supabase

Anything that takes more than about 2 s of LLM time goes onto a queue (post-session summaries, flashcard pack generation, the periodic topic scrape).

2. Reliability

2.0 Service Level Objectives

SLI	SLO	Window
API availability (1 - rate(5xx))	at least 99 percent	Rolling 5 min
<code>/api/messages</code> p95 latency	under 2 s	Rolling 5 min
<code>/api/topics</code> p95 latency	under 500 ms	Rolling 5 min
<code>/readyz</code> availability	100 percent	Continuous
Queue job success rate	at least 95 percent	Per day
Scraper freshness	at least 5 new topics	Per day

Full k6 measured results are in [docs/slo-table.md](#). Headline: at 50 VUs the error rate dropped from 6.81 percent to 0.27 percent by adding two gateway replicas, while meeting the latency SLO at baseline (0.34 s p95 against the 2 s target).

2.1 Reliability primitives

Five mechanisms, each pointed at a specific failure mode.

Primitive	Code location	Failure addressed
Circuit breaker (opossum) on Groq	<code>apps/gateway/src/plugins/groq.ts</code>	Groq slowdown or outage
Per-user rate limit (Redis backed)	<code>apps/gateway/src/server.ts</code>	Abusive client flooding the gateway
BullMQ retry with exponential backoff	<code>apps/workers/src/lib/queues.ts</code>	Transient worker or downstream failure
Dead-letter logging	<code>apps/workers/src/index.ts</code>	A job failing past its retry budget
Scraper idempotency key in Redis	<code>apps/workers/src/jobs/scrapeDuplicate.ts</code>	Duplicate work after a worker restart

2.2 Circuit breaker measured behaviour

We exercised the breaker with a mock Groq server (`mocks/groq-mock.mjs`) in slow mode (30 s sleep per request, well past the 15 s breaker timeout). A driver script (`mocks/breaker-driver.mjs`) fired 60 requests at concurrency 12. Full output is in `load/breaker-demo-output.txt`.

Phase	Calls	Per-call latency	Status
Closed (probing)	1 to 7	15.3 s (timing out against slow upstream)	breaker collecting failure samples
Transition	call 7		volumeThreshold (10) plus 50 percent error rate crossed, breaker opens
Open (short-circuit)	8 to 60	12 to 58 ms	requests fail fast, never reach upstream

The latency cliff is the headline: 15,300 ms to 25 ms between adjacent calls. Without the breaker, all 60 requests would have queued against the slow upstream and paid the 15 s timeout each. With it, only 7 calls actually paid that cost. The other 53 cost essentially zero CPU and never opened a socket to Groq.

Breaker state is exposed as a Prometheus gauge (`groq_circuit_breaker_state`), so the closed to open to half-open transitions show up live on the Grafana dashboard during the demo.

2.3 Other primitives

- **Rate limit.** 60 requests/minute per authenticated user (per IP otherwise). Counters live in Redis (`@fastify/rate-limit` Redis store), so the limit is shared across gateway replicas. No sticky sessions. Health probes are allowlisted.

- **Retries.** All BullMQ workers use `attempts: 3`, `backoff: { type: 'exponential', delay: 5000 }`. After three failed attempts the job is logged at `error` with `dlq: true` and shows up in Bull Board at `:3002/queues`.
- **Idempotency.** The scraper sets `scraper:slot:<date>:<6h-window>` in Redis on success with TTL 6 h. A second invocation in the same window is a no-op, so a worker crash mid-scraper does not cause duplicate topics on retry.

2.4 Recovery Time Objectives

Derived from the configured timeouts and retry policies, not measured separately.

Failure mode	Total RTO
Groq slowdown or outage	about 90 s (breaker opens within one 60 s window, then 30 s <code>resetTimeout</code> , then 2 s <code>/readyz</code> propagation)
Redis blip	about 20 s
Gateway replica crash	sub-second (nginx-lb round-robins to a healthy replica)
Worker job failure	about 20 s before DLQ visibility
Scraper duplicate replay	about 1 s

The Groq RTO dominates because it covers the user-facing chat path.

2.5 Tests in CI

The reliability story is pinned in CI, not just asserted in prose. The gateway has a Vitest suite (Fastify `inject()` harness, Redis, Supabase and Groq mocked at module boundaries) that pins the readiness contract: `/livez` returns 200, `/readyz` returns 200 when deps are up and `/readyz` returns 503 with the failing dependency named when Redis or Groq go down. CI runs `pnpm turbo test` ahead of typecheck and build on every push and PR. Source: [apps/gateway/src/tests/health.test.ts](https://github.com/ai-bridge/ai-bridge/blob/main/apps/gateway/src/tests/health.test.ts). Breaker, rate-limit and queue-retry tests are the natural next additions.

3. Scalability

The gateway scales horizontally. Workers and Redis are vertically bounded, on purpose (see Section 7).

3.1 Mechanism

```
docker compose up --scale gateway=3
```

An nginx load balancer (`nginx-lb`) sits in front and round-robins via Docker's built-in DNS. The web SPA and Prometheus both target `nginx-lb:3001`, so adding replicas needs no client-side or scrape-config change.

3.2 Measured scale-out behaviour

The k6 script at `load/gateway.js` runs three scenarios back to back: baseline (10 VUs for 30 s), stressed (ramp 0 to 50 VUs over 60 s, then taper) and scaled_out (50 VUs for 30 s). We ran it twice, once with one gateway replica and once with three, keeping everything else fixed.

Metric	1 replica	3 replicas	Δ
gateway_errors rate	6.81 %	0.27 %	−96 %
scaled_out p95 latency	2.43 s	2.15 s	−12 %
stressed p95 latency	1.73 s	1.84 s	+6 % (within noise)
Sustained throughput	39.3 req/s	39.4 req/s	unchanged

The error rate is where the scaling shows up. With one replica, the gateway dropped about 7 percent of requests under saturation. With three, that dropped to under 0.3 percent. Latency only improved slightly and throughput stayed flat.

Reading: under sustained load the gateway itself is the availability bottleneck. A single Node event loop cannot keep up with 50 concurrent VUs once you add auth and rate-limit checks, so it starts dropping connections. The downstream hot path (Supabase RTT, Redis round-trips) is the latency bottleneck, and adding gateway replicas does not change the per-request work outside the gateway.

The persistent ~25 percent `http_req_failed` rate in both runs is the rate limiter doing its job. 50 VUs at ~10 req/s is roughly 500 req/min, well over the 60 req/min cap, so most of those “failures” are 429s.

3.3 Threats to validity

- **Co-located load generator.** k6 ran on the same host as Docker Desktop, competing for CPU. Most visible in the unexpected **baseline** p95 regression on the 3-replica run.
- **Mocked Groq.** All Groq calls returned in under 1 ms. Deliberate (it isolates the gateway and queue layer) but it means absolute p95 numbers are not production estimates.
- **One iteration per configuration.** No statistical variance.

3.4 Production deployment

Production splits the system across two hosts. The gateway, workers, Redis and `nginx-lb` run on a single Oracle Cloud VM via [docker-compose.prod.yml](#), pulling multi-arch (`linux/amd64` and `linux/arm64`) images from GHCR. The React SPA runs on Vercel.

Three GitHub Actions workflows handle the pipeline, separated by path filter so a docs-only or web-only change skips the backend rebuild.

- [ci.yml](#). `pnpm turbo test` plus typecheck plus build on every push and PR. No deploy.
- [deploy.yml](#). Push to `main`, `paths-ignore` for web/docs/markdown. Runs the same checks, multi-arch `docker buildx` push to GHCR, SSH into the Oracle host for `git reset --hard origin/main`, `docker compose pull` && `up -d`. Then explicitly restarts `nginx-lb` so it re-resolves the new gateway container (Docker’s embedded DNS otherwise caches the old IP).
- [deploy-web.yml](#). Push to `main`, `paths` for web/packages/vercel.json. Runs the same checks then `npx vercel --prod`. The Vercel build ([vercel.json](#)) rewrites `/api/:path*` to the Oracle backend, so the browser only ever talks to the Vercel origin. No CORS at the gateway.

The prod compose omits Prometheus, Grafana and Bull Board (they remain in the local `docker-compose.yml` for the observability story) and the SPA (Vercel handles that).

4. Observability

We went with metrics and structured logs. Distributed tracing was a scope call, see Section 7.

All services emit single-line JSON via `pino`. Every line carries `service`, a `requestId` (from `X-Request-Id` if present, generated otherwise) and the relevant domain identifiers (`sessionId`, `jobId`, `queue`). Log level is tunable via `LOG_LEVEL`.

Both the gateway and the workers expose `/metrics` in Prometheus text format, scraped every 15 s.

Metric	Type	Labels	Where
<code>http_request_duration_seconds</code>	Histogram	method, route, status_code	gateway
<code>groq_request_duration_seconds</code>	Histogram	operation, status	gateway
<code>groq_circuit_breaker_state</code>	State	breaker	gateway
<code>bullmq_job_duration_seconds</code>	Histogram	queue, status	workers
<code>jobs_dead_letter_total</code>	Counter	queue	workers
<code>queue_depth_total</code>	Gauge	queue	gateway, workers
<code>active_sessions_total</code>	Gauge		gateway

Node defaults (event-loop lag, GC, RSS) are collected via `collectDefaultMetrics`. A Grafana dashboard provisioned from `docker/grafana/dashboards/edge-coach.json` auto-loads at `localhost:3000` with panels for request rate, p50/p95/p99 latency, 5xx rate, breaker state, queue depth and active sessions.

Two health endpoints back container orchestration. `GET /livez` always returns 200. `GET /readyz` pings Redis, Supabase and Groq with 2 s timeouts and returns 503 with the failing dependency named in the body.

5. Operator Surface — Agentic MCP

On top of the observability stack we added an operator surface: two small [Model Context Protocol](#) servers that let an agent (Claude Desktop, Claude Code or any other MCP client) inspect and remediate the running system without anyone having to SSH in or redeploy. The loop is detect (Grafana, alerts) then diagnose (`observability-mcp`) then remediate (`remediation-mcp`).

The two servers are split by privilege.

Server	Surface	Tools	Auth
observability-mcp	Read only over <code>/metrics</code> , <code>/readyz</code> and the safety policy file	<code>get_service_health</code> , <code>get_queue_metrics</code> , <code>get_circuit_breaker_state</code> , <code>get_active_sessions</code> , <code>get_groq_latency</code> , <code>get_safety_policy</code>	None (stdio, locally scoped)
remediation-mcp	Guarded mutations on Redis and the gateway admin route	<code>pause_queue</code> , <code>resume_queue</code> , <code>reset_circuit_breaker</code> , <code>flush_dead_letter_queue</code>	Layered: <code>zod.enum</code> queue allow-list, <code>ADMIN_API_KEY</code> for breaker reset, <code>confirm: true</code> for the irreversible flush, structured stderr audit log on every call

When the breaker demo trips the chat breaker, recovering becomes a three-step dialogue: call `get_circuit_breaker_state` to confirm the breaker is open, check that Groq is healthy again, then call `reset_circuit_breaker` to close it. Every action leaves an audit line on stderr (stdout is reserved for the MCP transport).

This does not replace distributed tracing. There are still no cross-service spans. But it does give us a typed way to ask the live system what is happening, which is the main thing we would otherwise be reaching for tracing to do.

Sources: [apps/observability-mcp/src/index.ts](#), [apps/remediation-mcp/src/index.ts](#), [apps/gateway/src/routes/admin](#)

5.1 Capability classification

Every capability the agent can touch falls into deterministic automation, agentic decision making or human in the loop. The line is not impact magnitude, it is whether the decision needs interpretation across heterogeneous signals. A deterministic threshold (the breaker trip) is fully automated. Synthesising `get_service_health`, the breaker gauge and the Groq latency histogram into “is Groq healthy enough to reset” is agentic, because no single metric answers it. Anything irreversible is human in the loop, regardless of how confident the agent is.

Capability	Class	Approval
Breaker trip, BullMQ retry, rate limiter, scraper idempotency	Deterministic	None
<code>get_service_health</code> plus <code>get_groq_latency</code> synthesis	Agentic (advisory)	None (read only)
<code>pause_queue</code> / <code>resume_queue</code> during a drain	Agentic (bounded)	None (reversible, typed enum)
<code>reset_circuit_breaker</code>	Human in the loop	Holder of <code>ADMIN_API_KEY</code>
<code>flush_dead_letter_queue</code>	Human in the loop	Explicit <code>confirm: true</code> from a human

Capability	Class	Approval
Infrastructure scaling	Operator only	Out of agent scope

Full reasoning and approval matrix are in [docs/safety-policy.md](#).

5.2 Guardrails

Three layers protect the system from a confused agent the same way they protect it from a confused operator.

1. **Type-level validation at the MCP boundary.** Zod schemas reject hallucinated tool names and malformed arguments before dispatch. The agent cannot execute a hallucination because the boundary refuses to dispatch one.
2. **Verify before act.** Before `reset_circuit_breaker` runs, the agent calls `get_service_health` and confirms the dependency is healthy. If not, the action is refused.
3. **Privilege split.** `observability-mcp` has zero mutation tools. An agent connected only to it cannot remediate at all.

Economic guardrails cap the agent at 10 tool calls per incident, 20 per principal per 10 minutes and about \$0.50 of inference cost per incident. Past any of these, the agent stops and escalates. The numbers and the per-month math are in [docs/cost-roi.md](#).

When telemetry is inconsistent, when the per-incident ceiling is hit or when the DLQ contains a poison-pill job with no clear root cause, the default is bounded reversible action (pause the queue, escalate) rather than speculative remediation (flush the queue). Full rules in [docs/safety-policy.md](#).

6. Safety policy and cost analysis

Full detail is in [docs/safety-policy.md](#) and [docs/cost-roi.md](#). The policy is also exposed to the agent through the `get_safety_policy` tool on `observability-mcp`, so the agent reads it at session start and self-binds before any remediation. Headlines:

- **Approval matrix.** Reversible actions (pause queue, breaker reset) can be agent autonomous or operator approved. Irreversible actions (DLQ flush, data deletion, infrastructure scaling) need a human in the path. The line is reversibility.
- **Audit by default.** Every remediation tool call writes a structured pino line on stderr with timestamp, tool, arguments and result.
- **Current OpEx.** \$0 to \$1/mo. Everything runs on free tiers (Oracle Always Free, Vercel hobby, Supabase free, GitHub Actions, a few cents of Groq for dev calls).
- **Projected OpEx at 20 users.** About \$2 to \$4/mo. Same free tiers plus around \$2 to \$3/mo of Groq. Free tiers stop covering us roughly at 200 to 500 users, mainly because of Supabase egress.
- **Cost of one hour of downtime.** Zero now. About \$20 to \$60/h projected at 100 paying users, dominated by churn risk.
- **Return on Agent.** About 6 minutes of MTTR saved per incident. At 4 incidents a month and a \$30/h ops rate, that is about \$12/mo saved against \$2 to \$5/mo of agent inference cost. 3 to 5 times ratio plus an automatic audit trail.

- **Budget compliance under stress.** No autoscaling, so cost cannot run away. A 6x traffic spike is absorbed by the rate limiter (429s).
- **Cost vs reliability trade-off.** 3 retry attempts on BullMQ workers, not 10. A poison-pill at 10 attempts consumes 85 minutes of retry budget.
- **Automation vs safety trade-off.** `pause_queue` is agent autonomous (reversible). `flush_dead_letter_queue` requires `confirm: true` from a human (irreversible).

7. Trade-offs

Full reasoning is in the ADRs.

Decision	Alternative	Why
Node.js workers	Go microservice	Shared TypeScript types, same toolchain, faster iteration
BullMQ on Redis	RabbitMQ or Kafka	One Redis instance does both queueing and rate limiting
Supabase	Self-hosted Postgres plus bespoke auth	Saved weeks on auth, cost is an external dependency
prom-client only	Full OpenTelemetry	Faster to instrument, fewer services. Cost is no cross-service tracing

The biggest one is the last. A more thorough submission would include OTel traces showing a request span propagated from gateway through the BullMQ job to Groq. We have it in future work. The MCP operator surface in Section 5 closes part of the gap by making live state queryable, but it is not a real substitute.

8. Future Work

- **OpenTelemetry traces.** Auto-instrument Fastify, ioredis and Prisma. Carry trace context through the BullMQ payload so the async path lands in the same trace.
- **Worker autoscaling.** Workers are a fixed count right now. BullMQ already supports multiple consumers, so this is mostly wiring and manifests.
- **Kubernetes manifests with HPA.** Lift the compose setup into a Helm chart with an HPA driven by `queue_depth_total` and `http_request_duration_seconds`.
- **Statistical load testing.** Multiple iterations per configuration with confidence intervals, run from a dedicated load-generation host.
- **Tighten remediation-mcp auth.** `pause_queue` and `resume_queue` currently rely on stdio locality and the zod enum. Production should require `ADMIN_API_KEY` for everything that mutates state.

See the repository for the full ADRs ([docs/adr](#)), the C4 diagrams ([docs/c4-diagram.md](#)), the SLO table ([docs/slo-table.md](#)), the operational safety policy ([docs/safety-policy.md](#)), the cost and ROA analysis ([docs/cost-roi.md](#)) and the raw k6 and breaker outputs ([load/](#)).